

## 2.5 Evaluation Criteria of Algorithm

کسی الگورتھم کو تین معیارات پر پرکھا جاتا ہے: درستگی (Correctness) یعنی درست نتیجہ دے، غیر معمولی صورتوں کو سن...

CS XI

Chapter 2: Computational Thinking &amp; Algorithm

Lecture 10 of 11

## 2.5 Evaluation Criteria of Algorithm

### LEARN IT

Many algorithms can solve the same problem. To judge which is better, we evaluate them on three criteria: **correctness**, **efficiency** and **clarity**.

### 2.5.1 Correctness

An algorithm is **correct** if:

- its output matches the expected outcomes;
- it handles **edge cases** gracefully (unusual inputs such as an empty list, 0, a negative number, or an item that is not present);
- it is guaranteed to finish — no infinite loops.

We test correctness with a **trace table** (dry run): we feed sample inputs and record every value by hand, then compare the result with what we expect.

### WORKED EXAMPLE

**Trace table example: Even/Odd checker.** Rule: divide the number by 2; if the remainder is 0 it is Even, otherwise Odd.

| Input | Operation (divide by 2, find remainder) | Remainder == 0? | Result |
|-------|-----------------------------------------|-----------------|--------|
| 3     | 3 / 2 (Quotient = 1, Remainder = 1)     | No              | Odd    |
| 6     | 6 / 2 (Quotient = 3, Remainder = 0)     | Yes             | Even   |
| 15    | 15 / 2 (Quotient = 7, Remainder = 1)    | No              | Odd    |

All three results match what we expect, so the algorithm is correct for these inputs. The same check in Python:

```
for number in [3, 6, 15]:
    remainder = number % 2
    if remainder == 0:
        print(number, "remainder", remainder, "→ Even")
    else:
        print(number, "remainder", remainder, "→ Odd")
```

### OUTPUT

```
3 remainder 1 → Odd
6 remainder 0 → Even
15 remainder 1 → Odd
```

A good tester would also try the edge case 0 (remainder 0 → Even, which is correct).

## 2.5.2 Efficiency

We measure algorithm efficiency through **time** (speed) and **space** (memory). The key question is: how fast does it run as the input size grows — does it **scale** well?

Computer scientists describe growth with **Big O notation**. It counts roughly how many steps are needed for N items:

- $O(1)$  — constant: same time, whatever N is.
- $O(\log N)$  — logarithmic: grows very slowly (Binary Search).
- $O(N)$  — linear: double N, double the time (Linear Search).
- $O(N^2)$  — quadratic: double N, four times the time (Bubble and Selection Sort).

### WORKED EXAMPLE

#### How the worst-case number of comparisons grows

| N (items) | Linear Search (N) | Binary Search (about $\log_2 N$ ) | Bubble/Selection Sort $N(N-1)/2$ |
|-----------|-------------------|-----------------------------------|----------------------------------|
| 10        | 10                | 4                                 | 45                               |
| 100       | 100               | 7                                 | 4,950                            |
| 1,000     | 1,000             | 10                                | 499,500                          |
| 1,000,000 | 1,000,000         | 20                                | 499,999,500,000                  |

This is why Binary Search is called excellent for Big Data and the simple sorts are called useless for it.

## 2.5.3 Clarity (Readability)

An algorithm must be easy to read, understand and modify. It should use **clear steps**, **meaningful variable names** and **good indentation**.

### Unclear

```
a=[3,6,15]
for x in a:
    if x%2==0:print(x,"E")
    else:print(x,"O")
```

### Clear

```
numbers = [3, 6, 15]
for number in numbers:
    if number % 2 == 0:
        print(number, "Even")
    else:
        print(number, "Odd")
```

Both work, but the second is far easier for another person (or you, a month later) to understand and change.

### REMEMBER IT

Remember **C-E-C**: **C**orrect (right answer, always finishes), **E**fficient (fast and light), **C**lear (easy to read).

### COMMON MISTAKE

Testing only "normal" inputs is not enough to prove correctness. Always include edge cases in your trace table, e.g., the first item, the last item, a missing item, or an empty list in a search.

### اردو میں سمجھیں

کسی الگورتھم کو تین معیارات پر پرکھا جاتا ہے: درستگی (Correctness) یعنی درست نتیجہ دے، غیر معمولی صورتوں کو سنبھالے اور ہمیشہ ختم ہو؛ کارکردگی (Efficiency) یعنی کم وقت اور کم میموری استعمال کرے؛ اور وضاحت (Clarity) یعنی پڑھنے، سمجھنے اور تبدیل کرنے میں آسان ہو۔ درستگی جانچنے کے لیے ٹریس ٹیبل (Trace Table) بنایا جاتا ہے۔

#### CHECK YOURSELF

1. Name the three evaluation criteria of an algorithm.
2. Give one example of an edge case for a search algorithm.
3. Efficiency is measured in terms of which two resources?

---

Answers: 1. Correctness, efficiency, clarity. 2. The item is not in the list (or the list is empty, or the item is the last element). 3. Time (speed) and space (memory).

## Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

**MCQ 1.** Correctness means:

- a. Output matches the spec, including edges
- b. The code is short
- c. It uses XOR
- d. It prints Urdu

**MCQ 2.** Time efficiency is:

- a. How fast it grows with  $n$
- b. Font size
- c. Cable length
- d. Number of comments

**MCQ 3.** Space efficiency is:

- a. Extra memory used
- b. Screen brightness
- c. TCP window only
- d. Pie slices

**MCQ 4.** An edge case for divide is:

- a. Divisor 0
- b.  $n=100$  always
- c. Color of GUI
- d. Sprint length

**MCQ 5.** Readability is part of:

- a. Quality / clarity, not Big O itself
- b. Physical layer
- c. NAND grouping
- d. Variance

**MCQ 6.** You evaluate an algorithm before coding so you can:

- a. Pick a good one
- b. Skip tests
- c. Avoid variables
- d. Draw Figma only

## Extra short questions

1. List four evaluation criteria.
2. What is an edge case?
3. Correct but slow — is it acceptable?
4. How do you show correctness in XI/XII?

## Extra long question

1. Compare linear vs binary search using correctness, time, space, preconditions and a 10,000-name example.

## Extra answer key

**MCQs:** 1. A 2. A 3. A 4. A 5. A 6. A

### Short

1. Correctness, time, space, clarity/simplicity (and sometimes completeness).
2. An extreme input: empty list,  $n=1$ , already sorted, not found, 0.
3. For tiny  $n$  yes; for huge  $n$  you need a better Big O.
4. Trace table or stepwise reasoning plus test cases.

### Long

1. Both can be correct. Linear  $O(n)$  any order; binary  $O(\log n)$  needs sort. Space  $O(1)$ . 10k names: ~10k vs ~14 comparisons. Mention the cost of keeping the list sorted.